Object-oriented programming

Tutorial 1. Classes and objects (Part2)

Exercise 4:

The attendance service of a training school wants to automate the management of student absences in training workshops. A student is characterized by their first name, last name, student ID, and an absence counter. Each training program is identified by a unique code (e.g., Or11), has a name, a list of enrolled students, and the workshops that have taken place. We assume that a student is enrolled in only one training program at a time, and a training program cannot have more than 30 students and 10 workshops. A workshop for a given training program is held on a specific date and time (e.g., workshop for the Or11 training program on 16/02/2025 at 14:30). After each workshop, the instructor submits the list of absent students to the attendance service. The service records this list for the corresponding workshop and updates its status from planned to conducted.

The system should be able to:

- Display the list of enrolled students for each training program, along with their total number of absences.
- Display the list of workshops for a given training program.
- Display the list of conducted workshops for a given training program.
- Display the list of absent students for a given workshop.

Q1 Given the following main function, propose an object-oriented solution by identifying the necessary classes, their attributes, and their methods.

```
public class Main {

    public static void main(String[] args) {

         // Create an attendance service object
        AttendanceService service = new AttendanceService();

         // Create a training
        Training training = new Training("Or11", "Oracle");
        service.addTraining(training);

        // Create students
        Student s1 = new Student("Ali", "Ahmed", "S001");
        Student s2 = new Student("Sara", "Belaid", "S002");
        Student s3 = new Student("Omar", "Kacem", "S003");

        training.addStudent(s1);
        training.addStudent(s2);
        training.addStudent(s3);

        Workshop w1 = new Workshop("16/02/2025", "14:30");
        Workshop w2 = new Workshop("17/02/2025", "10:00");
        Workshop w3 = new Workshop("19/02/2025", "16:00");
        Workshop w4 = new Workshop("25/02/2025", "10:00");

        training.addWorkshop(w1);
```

Object-oriented programming

```
        training.addWorkshop(w2);
        training.addWorkshop(w3);
        training.addWorkshop(w4);

        Student[] absenteesW1 = {s1, s2};
        Student[] absenteesW2 = {s3};
        Student[] absenteesW3 = {s1};

        //The service records the workshop absentees and updates its status
        service.recordAbsences(w1, absenteesW1);
        service.recordAbsences(w2, absenteesW2);
        service.recordAbsences(w3, absenteesW3);

        // Display the asked information
        service.displayEnrolledStudents();
        service.displayWorkshops(training);
        service.displayWorkshops(training,"Conducted");
        service.displayAbsentees(w1);
        }
    }
```

Q2: The attendance service wants more details about student absences. It now requires a list of all workshops each student was absent from, along with the corresponding date and time.

a. Identify the necessary modifications to fulfill this requirement.
b. Implement your solution.

Q3. We want to restrict the display operations by using System.out.println only in the main method (highlighted in bold below). To achieve this, the display methods should return a String instead of being void.

a. Make the necessary modifications.
b. What is the benefit of this modification?

Note: To create a *String* incrementally, use the *StringBuilder* class and its *append* method to concatenate Strings efficiently (Refer to the Annex at the end of this document)

```
public class Main {
    public static void main(String[] args) {
        // Create an attendance service object
        AttendanceService service = new AttendanceService();
        // Create a training
        Training training = new Training("Or11", "Oracle");
        service.addTraining(training);
        // Create students
        Student s1 = new Student("Ali", "Ahmed", "S001");
        Student s2 = new Student("Sara", "Belaid", "S002");
        Student s3 = new Student("Omar", "Kacem", "S003");

        training.addStudent(s1);
        training.addStudent(s2);
        training.addStudent(s3);

        Workshop w1 = new Workshop("16/02/2025", "14:30");
```

```
        Workshop w2 = new Workshop("17/02/2025", "10:00");
        Workshop w3 = new Workshop("19/02/2025", "16:00");
        Workshop w4 = new Workshop("25/02/2025", "10:00");

        training.addWorkshop(w1);
        training.addWorkshop(w2);
        training.addWorkshop(w3);
        training.addWorkshop(w4);

        Student[] absenteesW1 = {s1, s2};
        Student[] absenteesW2 = {s3};
        Student[] absenteesW3 = {s1};

        //The service records the workshop absentees and updates its status
        service.recordAbsences(w1, absenteesW1);
        service.recordAbsences(w2, absenteesW2);
        service.recordAbsences(w3, absenteesW3);

        // Display the asked information
        System.out.println(service.displayEnrolledStudents());
        System.out.println(service.displayWorkshops(training));
        System.out.println(service.displayWorkshops(training,"Conducted"));
        System.out.println(service.displayAbsentees(w1));

        //Q2: displaying details about a student's absences
        System.out.println(service.displayStudentAbsencesDetails(s1));
    }
}
```

Q4. We now want to organize our code further by "unloading" the concern of preparing display text from our classes (*Training, AttendanceService, Workshop*), leaving them responsible only for managing data. We will improve our code organization by creating a dedicated class for reporting.

a. Create a new class named *AttendanceReporter*.
b. Move all text building code (the methods that use StringBuilder to create report strings) from the different classes into this new class inside methods.
c. These methods should take the necessary objects (such as AttendanceService, Training, or Workshop) as parameters to retrieve the information they need to format.
d. What is the benefit of this modification?

Note: Notice how the new version of Main given below remains responsible for the final System.out.println, but it no longer needs to know the details of how the strings are constructed.

```
public class Main {
    public static void main(String[] args) {
        // 1. Initialization of Services
     AttendanceService service = new AttendanceService();
     AttendanceReporter reporter = new AttendanceReporter(); // New class for reporting

        // 2. Data Creation
        Training training = new Training("Or11", "Oracle");
        service.addTraining(training);

        Student s1 = new Student("Ali", "Ahmed", "S001");
        Student s2 = new Student("Sara", "Belaid", "S002");
        Student s3 = new Student("Omar", "Kacem", "S003");

        training.addStudent(s1);
        training.addStudent(s2);
```

Object-oriented programming

```
        training.addStudent(s3);

        Workshop w1 = new Workshop("16/02/2025", "14:30");
        Workshop w2 = new Workshop("17/02/2025", "10:00");
        Workshop w3 = new Workshop("19/02/2025", "16:00");
        Workshop w4 = new Workshop("25/02/2025", "10:00");

        training.addWorkshop(w1);
        training.addWorkshop(w2);
        training.addWorkshop(w3);
        training.addWorkshop(w4);

        // 3. Recording Absences (Logic via Service)
        Student[] absenteesW1 = {s1, s2};
        Student[] absenteesW2 = {s3};
        Student[] absenteesW3 = {s1};

        service.recordAbsences(w1, absenteesW1);
        service.recordAbsences(w2, absenteesW2);
        service.recordAbsences(w3, absenteesW3);

        // 4. Reporting: Format via reporter, display via Main
        System.out.println(reporter.formatEnrolledStudents(service));
        System.out.println(reporter.formatWorkshops(training));
        System.out.println(reporter.formatWorkshops(training,"Conducted"));
        System.out.println(reporter.formatAbsentees(w1));
        System.out.println(reporter.formatStudentAbsenceDetails(s1, training));
        }
}
```

Below is an example of the console output generated by the *main* method for questions 3 and 4.

```
=== ENROLLED STUDENTS & TOTAL ABSENCES ===
Training: Oracle
 - Ali Ahmed | ID: S001 | Absences: 2
 - Sara Belaid | ID: S002 | Absences: 1
 - Omar Kacem | ID: S003 | Absences: 1

Workshops for: Oracle
 [ Conducted ] 16/02/2025 at 14:30
 [ Conducted ] 17/02/2025 at 10:00
 [ Conducted ] 19/02/2025 at 16:00
 [ planned ] 25/02/2025 at 10:00

Conducted Workshops for: Oracle
 [ Conducted ] 16/02/2025 at 14:30
 [ Conducted ] 17/02/2025 at 10:00
 [ Conducted ] 19/02/2025 at 16:00

Absentees for Workshop on 16/02/2025:
 - Ali Ahmed (ID: S001)
 - Sara Belaid (ID: S002)

Detailed absence history for: Ali Ahmed
 * Missed workshop on 16/02/2025 at 14:30
 * Missed workshop on 19/02/2025 at 16:00
```

Annex.The StringBuilder Class

When you need to create a *String* by combining multiple pieces of data (especially inside a loop), using *StringBuilder* is the professional standard in Java. Unlike a regular String, which is immutable (cannot be changed once created), a StringBuilder is mutable, making it much faster and more memory-efficient for building long reports.

1. Core Method: *append()*

The *append()* method adds text or data to the end of the current sequence. It can accept almost any data type (String, int, char, etc.).

Syntax:

```
public StringBuilder append(String str)
```

2. Common Methods

- new *StringBuilder()*: Creates an empty string builder.
- *append(data)*: Adds the data to the end.
- *toString()*: Converts the StringBuilder into a final String so it can be returned or printed.

3. Practical Example

In the *Training* class of Question 3, you can use it like this to build a list of workshops:

```
public String displayWorkshops() {
if (workshopCount == 0) return "No workshops for " + name;
StringBuilder result = new StringBuilder("Workshops for the training " +
this.name + " : \n");
for (int i = 0; i < workshopCount; i++) {
    result.append("- ").append(workshops[i].display()).append("\n");
}
return result.toString();
}
```